1. What is an LSTM?

A Long Short-Term Memory (LSTM) network is a specialized type of Recurrent Neural Network (RNN). While a standard RNN only has "short-term" memory (the hidden state), an LSTM introduces a "long-term" memory called the **Cell State**.

In the context of your prosthesis, the LSTM doesn't just look at the current force; it tracks the **trend** of the gait cycle. It uses "gates" (mathematical valves) to decide which force fluctuations are noise (to be forgotten) and which are indicators of gait phase transitions (to be remembered).

2. The Architecture: The Three Stages

An LSTM processes each millisecond of your force data through three gates.

Stage 1: The Forget Gate (f_t)

Intuition: Decides how much of the previous gait context to throw away. If the sensor detects a sudden drop in force, this gate might "forget" the stance-phase memory to prepare for the swing-phase.

Formula:

$$f_t = \sigma(W_f \cdot x_t + U_f \cdot h_{t-1} + b_f)$$

- σ: Sigmoid function (outputs 0 to 1).
- x_t: Your current inputs (F_z and $\dot{F}_z$).

Stage 2: The Input Gate (i_t) & Candidate (g_t)

Intuition: Decides what new information from the current force reading is worth adding to the long-term memory.

Formula (The Update):

$$i_t = \sigma(W_i \cdot x_t + U_i \cdot h_{t-1} + b_i)$$

$$g_t = \tanh(W_g \cdot x_t + U_g \cdot h_{t-1} + b_g)$$

Updating the Cell State (c_t):

$$c_t = f_t * c_{t-1} + i_t * g_t$$

(This is the "Conveyor Belt" where long-term memory is stored).

Stage 3: The Output Gate (o_t)

Intuition: Filters the long-term memory to produce the current "Short-term" hidden state, which is used to calculate knee velocity.

Formula:

$$o_t = \sigma(W_o \cdot x_t + U_o \cdot h_{t-1} + b_o)$$

$$h_t = o_t * \tanh(c_t)$$

3. Hyperparameters: The "Knobs" of the Model

To get the performance in your paper, you tuned specific "knobs" that control how the LSTM learns and behaves.

A. Hidden Dimension (Size: 32)

- **Description:** The number of parallel "memory cells" inside the LSTM.
- **Significance:** A size of 32 means the model can track 32 different features of the gait simultaneously (e.g., heel-strike detection, weight-shifting speed, etc.).
- **Effect:** Too few units lead to **underfitting** (can't learn the gait); too many lead to **latency** (too slow for real-time control).

B. Fully Connected (FC) Layer (Width: 8, 16, or 32)

- **Description:** A linear layer that sits *after* the LSTM.
- **Significance:** It acts as the "Interpreter." It takes the 32 abstract memory features and decodes them into a single physical value (Knee Velocity).
- **Formula:** $Velocity = W_{fc} \cdot h_t + b_{fc}$

C. Learning Rate (LR: 5×10^{-4} to 5×10^{-3})

- **Description:** The step size used during Gradient Descent.
- **Significance:** It determines how aggressively the model changes its weights when it makes a mistake.
- **Effect:** If too high, the model "overshoots" and becomes unstable; if too low, it takes forever to train.

D. L2 Regularization (Weight Decay: 1×10^{-3})

- **Description:** A penalty added to the loss function that discourages large weights.
- **Significance:** It forces the model to stay "smooth."
- **Formula:** $TotalLoss = MSE + \lambda \sum w^2$
- **Effect:** Prevents **Overfitting**. It ensures the model doesn't overreact to a single noisy sensor spike.

E. Batch Size (e.g., 64 or 512)

- **Description:** The number of 100ms windows processed before the model updates its internal weights.
- **Significance:** Larger batches provide a more stable "average" gradient, while smaller batches make the training more stochastic (random).

F. Epochs (25)

- **Description:** The number of times the model sees the entire dataset.
- **Significance:** In your paper, 25 epochs was the "saturation point" where the model stopped improving its accuracy without beginning to overfit.

4. Application-Specific Hyperparameter: Window Size (W)

While not a "tuning" parameter for the neurons themselves, the window size is the most important for prosthetics.

- **Window Size (100–150ms):**
  - **Short Window (<50ms):** Not enough "history" to tell if the user is at the start or end of stance.
  - **Long Window (>200ms):** Introduces "lag." The model is still thinking about what happened 200ms ago, making the prosthesis feel sluggish to the user.

By combining the **LSTM structure** (to handle time) with **Hyperparameter tuning** (to handle noise and complexity), you created a model that estimates velocity as accurately as a GP but with the temporal awareness of a human gait.

-**----

GP Regression

--> not a single function to fit the data; instead, entire distribution of possible functions.

***** Limited data and Uncertaninty --> GP

GP detemine probabiltiy not over individual data but over entire functions

---> GP is a collection of random variables where any finite subset follows a multivariate Guassian distribution

f(x) = GP(m(x), k(x,x'))

m(x) is mean function

k is kernel or covariance function, how points relate to eachother

Gaussian Process Regression (GPR) is a non-parametric, Bayesian approach to regression. Unlike a Neural Network (which learns fixed weights), GPR is a **memory-based learner**. It predicts the knee velocity by comparing current sensor inputs to a collection of previously "seen" examples.

- **The Key Advantage:** GPR provides not just a prediction (mean), but also an **uncertainty (variance)**. In prosthetics, this acts as a "safety sensor"—if the input is weird (e.g., a trip or sensor drift), the uncertainty spikes, allowing the controller to revert to a safe state.
- **The "Similarity" Philosophy:** It assumes that if Input A (Force) is close to Input B, the Output A (Velocity) should be close to Output B. This "closeness" is defined by the **Kernel**.

2. Mathematical Architecture

To make GPR work in real-time (1kHz) on a robotic leg, we use **Sparse GPR** with an **Active Set**.

A. The Components

1. **Active Set (X_{train}):** A 100×2 matrix. These are the 100 most "representative" points of a gait cycle chosen from your 300+ training strides. They are the landmarks of the gait cycle (Heel Strike, Mid-Stance, etc.).
2. **Alpha Vector (α):** A 100×1 vector of "Dual Weights." These are pre-computed in MATLAB. They represent the "contribution" of each active point to the final velocity, accounting for redundancies between the points.
3. **The Kernel (K):** The Matern 3/2 function. It defines the "smoothness" of the walking motion. It is physically appropriate for gait because it allows for the "sharp but continuous" force changes seen in human walking.

B. The Hyperparameters

- **Length-scale (l):** How far a known data point's influence reaches. A small l makes the leg "twitchy"; a large l makes the leg "sluggish."
- **Signal Variance (σ_f^2):** Scales the output to match the physical units (rad/s) of the knee velocity.
- **ARD (Automatic Relevance Determination):** Means we have a separate l for **Force** and **Force Derivative**, allowing the model to prioritize whichever sensor is more reliable for prediction.

3. The Implementation Logic (C++)

The real-time prediction follows a three-step pipeline:

Step 1: Kernel Comparison (The "Similarity Ballot")

For every new sensor reading (x_{obs}), we compare it to every point (x_i) in the Active Set.

$$k_i = \sigma_f^2 \left(1 + \frac{\sqrt{3}d}{l}\right) \exp\left(-\frac{\sqrt{3}d}{l}\right)$$

- This results in **k_***, a vector of 100 values between 0 and 1.
- If $k_i = 1$, the current step is identical to that memory.
- If $k_i = 0$, the current step is totally different.

Step 2: Weighted Sum (The "Final Vote")

The final prediction (y_*) is the dot product of the similarity vector and the Alpha weights.

$$y_* = \sum_{i=1}^{100} k_i \cdot \alpha_i$$

This is extremely fast in C++ because it is just a loop of 100 simple math operations.

Step 3: Variance/Safety (Optional)

If the sum of k_i is very low, it means the current sensor data looks like **nothing** in the training set. The leg can then trigger a "safe mode."

PID

$$u = \underbrace{k_p e}_{\text{spring}} + \underbrace{k_i \int e\,dt}_{\text{steady state error}} + \underbrace{k_d \dot{e}}_{\text{damper}}$$

```
For every ds {
    e = desired - read_sensor( )      → low pass
    edot = (e - eprev) / dt
    eint = eint x e*dt
    u = kp e + ki x eint + kd edot    → eint limit
    eprev = e
    send_control(u)                   → Limit
}
```

	order	Controller
Control mass → Force	2	PD, PID
Control V → C using current	1	P, PI
Control I → R using V	0	P, PI

k_i = improve steady state but can cause instability

$$\boxed{\text{Empirical Kevin Lynch}}$$

DC Motor Control

Layer 1: The Simple Switch and Flyback Diode

Imagine a circuit with a battery, a motor, and a switch. When you close the switch, the motor spins. But there is a hidden danger here.

- **The Inductive Load:** A motor is essentially a giant coil of wire (an inductor). Inductors **resist changes in current**.
- **The "Kick":** When you open the switch to stop the motor, the magnetic field inside the motor collapses instantly. This creates a massive voltage spike (Back EMF) that can be hundreds of volts. Without protection, this spike will create a spark that destroys your switch or fries your microcontroller.
- **The Flyback Diode:** We place a diode in parallel with the motor, pointing "up" (away from the ground).
  - When the motor is running, the diode does nothing.
  - When the switch opens, the diode provides a safe "loop" for that extra energy to circulate and dissipate as heat. **It is the "safety valve" of the circuit.**

Layer 2: PWM (Speed Control)

Now, instead of just "On" or "Off," you want the motor to spin at 50% speed. You could use a resistor to lower the voltage, but that would waste massive amounts of energy as heat. Instead, we use **PWM (Pulse Width Modulation)**.

- **The Concept:** You flip the switch on and off thousands of times per second (e.g., 20 kHz).
- **Duty Cycle:** This is the ratio of "On" time to "Off" time.
- **Effective Voltage:** The motor is a heavy mechanical object with inertia, so it cannot react to the individual pulses. It "feels" the **average voltage**.
  - If you have a 12V battery and a 50% duty cycle, the motor behaves as if it is receiving a steady 6V.

$$V_{avg} = V_{source} \times \text{Duty Cycle}$$

Layer 3: The H-Bridge (Directional Control)

A single switch can only make the motor spin in one direction. To reverse it, you would have to physically flip the battery around. The **H-Bridge** automates this. It uses four switches (usually MOSFETs) arranged like the letter "H."

- **Forward Path:** You close the top-left switch (S1) and the bottom-right switch (S4). Current flows from left to right through the motor.
- **Reverse Path:** You open S1/S4 and close the top-right (S3) and bottom-left (S2). Current flows from right to left.
- **Braking:** If you close both bottom switches (S2 and S4), the motor's terminals are shorted together. This creates a magnetic drag that stops the motor quickly (Dynamic Braking).
- **The "Shoot-Through" Danger:** If you accidentally close S1 and S2 at the same time, you create a direct short circuit from power to ground. This is called "shoot-through" and will instantly blow up the H-Bridge.

Why PI for Inner Loop

To answer your specific questions about the types of controllers (P, I, D) and why we use them in specific loops, let's break it down by the "layers" of the control stack.

1. The Inner Loop: Current (Torque) Control

Standard: Almost always a PI Controller.

You asked if it can be **PD**. In the current loop, we almost never use the Derivative (D) term. Here is why:

- **The Noise Problem:** Current is measured using shunt resistors or Hall-effect sensors. These signals are incredibly noisy. Since the D term calculates the "rate of change," it would amplify that high-frequency noise, causing the motor to "hum" or vibrate and potentially damaging the transistors in your H-bridge.
- **The Requirement:** The goal of the current loop is to reach the target current with **zero steady-state error**. The Integral (I) term is what ensures that the motor actually produces the exact torque requested by the computer.
- **Elmo Specifics:** Elmo drives use a high-speed **PI** loop for current, often running at 10 kHz to 20 kHz.

2. The Outer Loop: Position Control

Standard: P, PD, or PID.

In the position loop, the rules change. Here, the D term is very common.

- **Damping (The "Lead" Term):** You mentioned that D generates "lag." Actually, in control theory, the D term provides **Phase Lead**. It "predicts" that the robot is about to overshoot its target and applies a "braking" force. This adds **damping** to the system.
- **The "Lag" Misconception:** You are right that it can feel like "lag" because we have to **filter** the D term heavily to handle encoder noise. That low-pass filter is what introduces the actual phase lag.
- **When to use I:** We use the I term in position control if the robot is fighting gravity (like a heavy arm) and can't quite reach the final millimeter.

3. The Middle Loop: Velocity Control

Standard: PI or PIV.

Many professional systems insert a velocity loop between position and current.

- **PIV Control:** Elmo and other industrial drives often use a **PIV** structure. It uses a **Proportional** gain for position and a **PI** loop for velocity. This is often more stable than a standard PID for heavy robotic links.

4. The Path of the Signal (The Full Block Diagram)

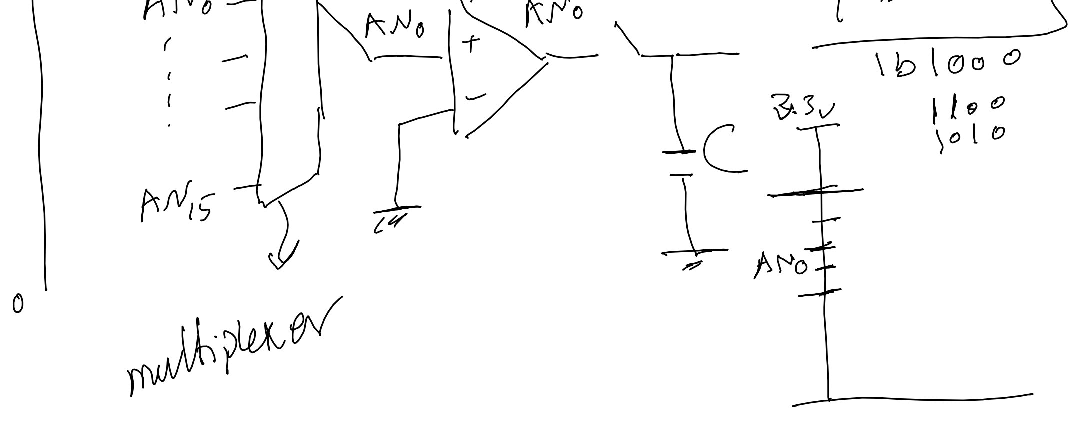
Here is how your data travels through the "Physical AI" stack:

1. **Motion Planner:** Generates the "Trajectory" (Target Position x_{ref}).
2. **Position Controller (P or PD):** Compares x_{ref} to the Encoder Feedback (x). It outputs a **Velocity Command** (v_{cmd}).
3. **Velocity Controller (PI):** Compares v_{cmd} to the calculated speed. It outputs a **Current Command** (I_{cmd}), which represents Torque.
4. **Current Controller (PI):** Compares I_{cmd} to the Actual Current measured by the drive. It outputs a **Voltage Duty Cycle**.
5. **PWM Generator:** Translates that voltage into "On/Off" pulses for the **H-Bridge**.
6. **H-Bridge:** Flips the MOSFETs to send high-power current to the **DC Motor**.
7. **Motor/Encoder:** Moves the robot and sends the signal back to Step 2 to close the loop.

💡 Summary Checklist for your Interview

If David asks about the loop structure, you can show off your deep understanding like this:

"I'm familiar with the **Cascade Control** architecture. In the **Inner Current Loop**, I would typically use a **PI controller** because it ensures zero steady-state torque error while avoiding the noise amplification issues of a D term. For the **Outer Position Loop**, I'd use **PD** or **PID** control, where the D term is essential for adding damping and preventing overshoot, especially in high-inertia robotic arms. In a system like an Elmo drive, I'd also look to tune the **Velocity loop** to ensure smooth motion before it ever reaches the current layer."



Modern Robotic

- Configuration Space:
  - all possible Configuration of a Robot
  - it can get parametrized either | implicitly
    | explicitly

explicit → parametrized use ~~the~~ # Dof

implicit → " use Euclidean sub to Constraint.

- Task
  - Robot task Can naturally expressed
    - ↳ focus on task Not the robot does the task

- Workspace → configuration of end-effect
  that the robot can reach

Nothing to do with particular task

* dexterous Workspace

→ set of pos that is possible orientation

---

chapter 3

implicit:

representation of configuration will not use a minimum set of coordinates

→ velocities will not be some derivative of coordinates

$$R_{sb} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} : 9 \text{ elements}$$

ref ↙ ↘ represented

Point → 3 possible or 3 doF

→ 6 holonomic constraints

| 3 column vectors = 1 unit vector

$R^T R = I$ ⟵ | 3 dot product of any column is 0

$\det R = 1$

$R^{-1} = R^T$, $\|Rx\| = \|x\|$

Rotation Matrix Usage :

1) Represent an orientation
2) change a frame of ref of < vector / frame
3) Rotate a vector or frame

1). $R_{sc} \Rightarrow R_{cs} = R^T_{sc} = R^{-1}_{sc}$

2) $R_{bc} \longrightarrow R_{sc} =$

$R_{sb} \times R_{bc}$

or vector : $P_b = [\;]$

$P_s = R_{sb} P_b = [\;]$

3) $R_{sb} = R = Rot(z, 20)$

$$\overline{R}_{sc'} = R \cdot R_{sc} \qquad R_{sc''} = R_{sc} R$$

Angular Velocity

orientation: R , Angular Velocity $= \dot{R}X$

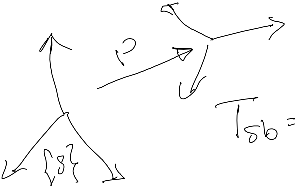
$$\omega_s = \hat{\omega}_s \dot{\theta} \rightarrow \begin{array}{l} \dot{\hat{x}}_b = \omega_s \times \hat{x}_b \\ \dot{\hat{y}}_b = \omega_s \times \hat{y}_b \\ \dot{\hat{z}}_b = \omega_s \times \hat{z}_b \end{array} \left\{ \begin{array}{l} x \times y = [x]y \\ x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^3 \\ [x] = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix} \\ [x] = -[x]' \end{array} \right.$$

$$\rightarrow \dot{R}_{sb} = [\dot{\hat{x}}_b, \dot{\hat{y}}_b, \dot{\hat{z}}_b] = [\omega_s] R_{sb}$$

$$\omega_b = R_{bs}\omega_s = R_{sb}^{-1}\omega_s = R_{sb}^{T}\omega_s$$

Transformation Matrix

$$T = \begin{bmatrix} R & P \\ 0 & 1 \end{bmatrix} \qquad T^{-1} = \begin{bmatrix} R^T & -R^T P \\ 0 & 1 \end{bmatrix}$$



$$T_{sb} = \begin{bmatrix} R_{sb} & P \\ 0 & 1 \end{bmatrix}$$

1) Represent a Configuration

2) change reference frame $T_{sc} = T_{sb} \times T_{bc}$

$$P_s = T_{sb} P_b \; X \rightarrow \begin{bmatrix} P_s \\ 1 \end{bmatrix} = \begin{bmatrix} T_{sb} \end{bmatrix} \begin{bmatrix} P_b \\ 1 \end{bmatrix}$$

3) displace vector / frame

$$\underline{\text{Twist}} = \mathcal{V} = \begin{bmatrix} S_\omega \\ S_v \end{bmatrix} = S\dot{\theta}$$

$$T = \begin{bmatrix} R & P \\ 0 & 1 \end{bmatrix} ; \; [Ad_T] = \begin{bmatrix} R & 0 \\ [P]R & R \end{bmatrix}$$

$$V_b = [Ad_{T_{ba}}] V_a$$

Forward Kinematics

$$e^{[\omega]\theta} = \text{Rot}(\hat{\omega}, \theta) =$$

$$I + \sin\theta\,[\omega] + (1-\cos\theta)[\omega]^2$$

$$[\omega_b] = R^{-1}\dot{R}$$

$$[\omega_s] = \dot{R}R^{-1}$$

$$[\mathcal{V}_b] = \begin{bmatrix} [\omega_b] & v_b \\ 0 & 0 \end{bmatrix}$$

$$[\mathcal{V}_s] = \begin{bmatrix} [\omega_s] & v_s \\ 0 & 0 \end{bmatrix}$$

$$S = \begin{bmatrix} \omega \\ v \end{bmatrix} \rightarrow \text{Screw Axis}$$

$\omega \rightarrow$ Angular Velocity unit vector

$v \rightarrow$ linear Velocity

$$T_{\theta} = e^{[S_1]\theta_1} e^{[S_2]\theta_2} e^{[S_3]\theta_3} M$$

$$e^{[S]\theta} = \begin{bmatrix} \text{Rotation} & \text{Translation} \\ 0 & 1 \end{bmatrix}$$

Rotation (3x3)

$$e^{[S]\theta} = I + \sin\theta\,[\omega] + (1 - \cos\theta)\,[\omega]^2$$

Translation

$$\cancel{e^{[\omega]}}\ (I\theta + (1 - \cos\theta)[\omega] + (\theta - \sin\theta)[\omega]^2)v$$

Prismatic =

$$e^{S[\theta]} = \begin{bmatrix} I & v\theta \\ 0 & 1 \end{bmatrix}$$

Revolute =

$$e^{[S_1]\theta_1} = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 & 0 \\ \sin\theta_1 & \cos\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Prismatic =

$$\begin{bmatrix} 1 & 0 & 0 & \theta_2 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

General Formula:

$$e^{[S]\theta} = \begin{bmatrix} e^{[\omega]\theta} & G(\theta)v \\ 0 & 1 \end{bmatrix}$$

$$G(\theta) = I\theta + (1-\cos\theta)[\omega] + (\theta - \sin\theta)[\omega]^2$$

~~Pold~~

$$P_{new} = \overset{\downarrow}{q} + \mathrm{Rot}(\omega,\theta)_{a}(\underbrace{P_{old}}_{\text{the claw @ home}} - \underbrace{q}_{\text{pivot joint}})$$

Velocity kinematics

$$x(t) = f(\theta(t))$$

$$\frac{d}{dt}x(t) = \frac{d}{dt} f(\theta(t)) \rightarrow \dot{x} = \frac{\partial f}{\partial \theta}(\theta)\,\dot{\theta}$$

$$\rightarrow \underbrace{\frac{\partial f}{\partial \theta}}_{\text{Jacobian}} = \begin{bmatrix} \frac{\partial f_1}{\partial \theta_1} & \cdots & \frac{\partial f_1}{\partial \theta_n} \\ \vdots & & \\ \frac{\partial f_m}{\partial \theta_1} & \cdots & \frac{\partial f_m}{\partial \theta_n} \end{bmatrix} \in \mathbb{R}^{\underbrace{m}_{\text{end-eff coord}} \times \overset{\text{\# of joints}}{\overset{\uparrow}{n}}}$$

Singularity : Dimension of column space
Jacobian drops from its Max Value

* Use Jacobian to find limit of end-effector velocity due to limit of joint velocity

$$\dot{\theta}^T \tau = V_{tip}^T F \qquad \dot{\theta}^T \tau = [J(\theta)\dot{\theta}]^T F$$

$$\rightarrow V_{tip} = J(\theta)\dot{\theta} \rightarrow \qquad \dot{\theta}^T \tau = \dot{\theta}^T . J^T(\theta) . F$$

$$\rightarrow \boxed{\tau = J(\theta) . F}$$

$$\boxed{F_{tip} = J^{-T}(\theta)\tau}$$

* Use J^T or J^{-1} to find force limits

Space Jacobian

$$V_s = J_s(\theta)\dot{\theta} = [J_{s_1}(\theta) \; J_{s_2}(\theta)]\dot{\theta}$$

$$J_{s_3} \leq [\theta] = 0 \rightarrow J_{s_3} = S_3 \xrightarrow{then} T_{ss''} = e^{[S_1]\theta_1} \cdot e^{[S_2]\theta_2}$$

$$\rightarrow J_{S_3}(\theta) = [Ad_{T_{ss''}}] \cdot S_3$$

$$Ad_T = \begin{bmatrix} R & 0 \\ [p]R & R \end{bmatrix} \in \mathbb{R}^{6 \times 6}$$

Body Jacobian

$$T_{b''b} = e^{[B_4]\theta_4} e^{[B_5]\theta_5}$$

$$T_{bb''} = \left(e^{[B_4]\theta_4} e^{[B_5]\theta_5} \right)^{-1} =$$

$$e^{-[B_5]\theta_5} e^{-[B_4]\theta_4}$$

$$J_{b3}(\theta) = [Ad_{T_{bb''}}] \cdot B_3$$

$$J_b = [Ad_{T_{bs}}] J_s \quad \text{and} \quad J_s = [Ad_{T_{sb}}] J_b$$

Force & Torque

$$\tau = J_b^T(\theta) F_b$$

Manipulability

$$v_{tip}^T A^{-1} v_{tip} = 1$$

$A = JJ^T \longrightarrow$ velocity ellipsoid

$A = (JJ^T)^{-1} \longrightarrow$ force ellipsoid

velocity $\leftarrow \underset{\text{inverse}}{(-1)} \leftarrow$ force

- only small forces can be applied where the larger velocity is in that direction

→ only small velocity is allowed where the large forces were applied

$$\mu_1 = \sqrt{\frac{\lambda_{max}(A)}{\lambda_{min}(A)}} \geq 1 \quad (\lambda = 1 \text{ isotropic}) \quad \uparrow\uparrow \text{ Singularity}$$

$$\mu_2 = \frac{\lambda_{max}}{\lambda_{min}} \geq 1$$

$$\mu_3 = \sqrt{\lambda_1 \lambda_2 \cdots} = \sqrt{\det(A)}$$

J_{bw} → Ang vel / Moment ellipsoid

J_{bv} → Linear vel / Moment ellipsoid

Inverse kinematic

numeric Solution

↳ Newton_Raphson

$$x_d - f(\theta^i) = J(\theta^i)\,\Delta\theta$$

$$\Rightarrow \Delta\theta = J^{-1}(\theta^i)\left[x_d - f(\theta^i)\right]$$

① Given x_d → initial guess θ^0

③ $e = x_d - f(\theta^i)$ → $\|e\| > \varepsilon$ ↓

$$\theta^{i+1} = \theta^i + J^{\dagger}(\theta^i) e$$

increment i

In Body Jacobian

$$T_{bd}(\theta^i) = T_{sb}^{-1}(\theta^i) T_{sd} = T_{sb}(\theta^i) T_{sd}$$

$$[\mathcal{V}_b] = \mathrm{Log}\, T_{bd}(\theta^i)$$

in Newton Raphson ↓

$$[\mathcal{V}_b] = \log\left(T_{sb}^{-1}(\theta^i) T_{sd}\right)$$

Modern Robotic

- Configuration Space:
- all possible configuration at a Robot
- it can get parametrized either/implicitly

explicitly explicit → parametrized use me#Dot implicit → is use Euclidin sub to

constant.

- task
- Rabat task Can naturally expressed ↳ focus on task Not the robot does

the task

- Workspace → configuration of end-ether

that the robot can reach Nothing to do with particular task

- dexterous Workspace
- set at pas. that is possible mentation

chapter}implicit:

representation of configuration will not use a minimum set of coordinates

- velocities will not be same derivative of coordinates
- 1:], I elementsRsb - ! % pants} possible

w} doF not represented

- € holonomic constraint

? column Vectors = I wit veto RTR = I

3 dot pandect of any column is? det E =! R-E RT, HR ✗ 11 = 11×11

Rotation Matrix Usage ..

vector 1) Represent an orientation 2) change a home of net of & home3) Rotate a vector or frame

1) . Rsc ⇒ Resz Rtsc-Rtsc

2) Rbc → Rsc =

Rsbx Rbc

or vector: Ps = [I

Ps = Rsb Pb. = []

3) R sb-R-Rot (z, 20)

=

Rse' - R. Rsc Rsc"-RscR

Angular Velocity

orientation: E, Angular Nality-RX

Ws-↳ ♂ → %_us ✗ Xb ✗ ✗ Ya [XJY

^ I, = Ws ✗ Yb

a. [&;) ER Éb=Ws ✗ Eb

♀, ⅓ ✗ 2 Ext. [

① -X,

- X2 ✗ I 0

LET =-Ex]'⅓, Y's, Eb]-[w] Rsb

- R=

Sb

Wb = Rbs Ws = Rfb Ws =P😉UsTransformation MatrixT. [? P] T" [RT -RTP

☐ L 2

Rsb P 1Tsb:

a IIs}1) Represent a Configuration

2) change retreneeframe Tseatsbel be

Tsb) [BYPs = TsbB × → (? Z3) displace Vector/frame

Test-N-E:/⇐ so

R ° R P

> EAT.

CDR R]=

•

Vb = [AdTba] Va

Forward kinematics

&"-Rot (w̄, 8) a

I + since [w] + Cl-case) [WT

Lab] = R" I

Cws], ER"

Cub] Nb [Vb] =

•

[us] a [Cars] Vs

- °

Angular Velocity

unit vector 5. [11-mar

Axis inier velocity

[s.IQ [52302 13103 µ To = e e

[SO, [Rotation Translation @ ∘ $

Rotation (3×3)

e' "= I-5in@In] + Cl-costs [w]²

Translation

etCLO-CL-GSGCWJ-CO-s.no) Cor'JN

prismatin.

I no]es💔I.

- IRevolate.

8650, _side 0

:Esso,

cash ∘ sind, T

☐

- 0

G O

Prismatic = 1 a ∘ on

☐ 1 O O

☐ O

☐ 6

General formula:

e". easo Goin]

I

☐

God. Id + (L-Cas#Car] +

(O- Smid] Cut'2- oldP new' 9 + Rotcw.ae?.iP.id. ♀)

↘ the claw@home protfont

Yedaily kinematicsActs =f (Oct))

- acts. d- Icom) → i = afford

It ⇒ stints of

son epman

- 8£. so, of

Stm - Id end-eff

son

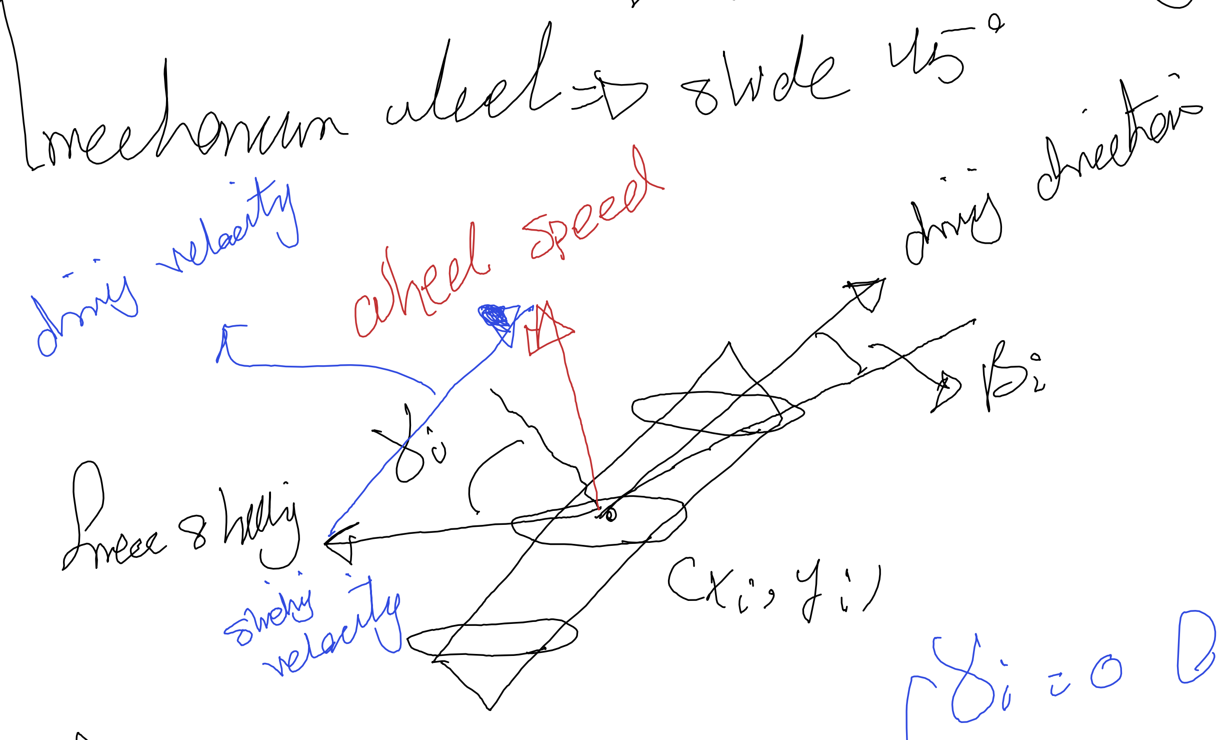
GordSO,JacobianSingularity: Dimension at calom space

Jacobin drops homits Max value

- Use Jacobian to hid lint at end-effector velocity due to limit of joint velocity

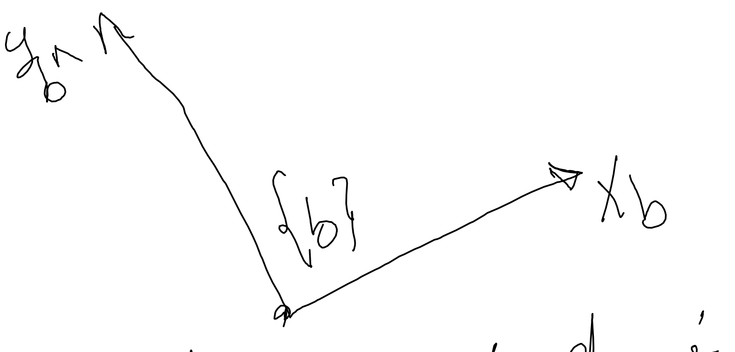
Wheeled Mobile Robot

- * Conventional wheel ⇒ sideway sliding
- * Omniwheel ⇒ slide 90°
  - → allow sideway sliding
- mechanum wheel ⇒ slide 45°


driving direction
driving velocity
wheel speed
γ_i
β_i
free sliding
sliding velocity
(x_i, y_i)

$\gamma_i = 0$ Omni

$\gamma_i = 45$ Mecanum



fixed to Robot chassis

wheel driving speed: rotation speed of Motor Attach to wheel

$$u_i = \underbrace{\underbrace{\frac{1}{r_i}\begin{bmatrix}1 & \tan\gamma_i\end{bmatrix}}_{\text{Component in driving dir}}\underbrace{\begin{bmatrix}\cos\beta_i & \sin\beta_i\\ -\sin\beta_i & \cos\beta_i\end{bmatrix}\begin{bmatrix}-y_i & 1 & 0\\ x_i & 0 & 1\end{bmatrix}\mathcal{V}_b}_{\text{linear velocity @wheel, in wheel frame}}}_{h_i(\phi),\ 1\times 3\ \text{row vector}}$$

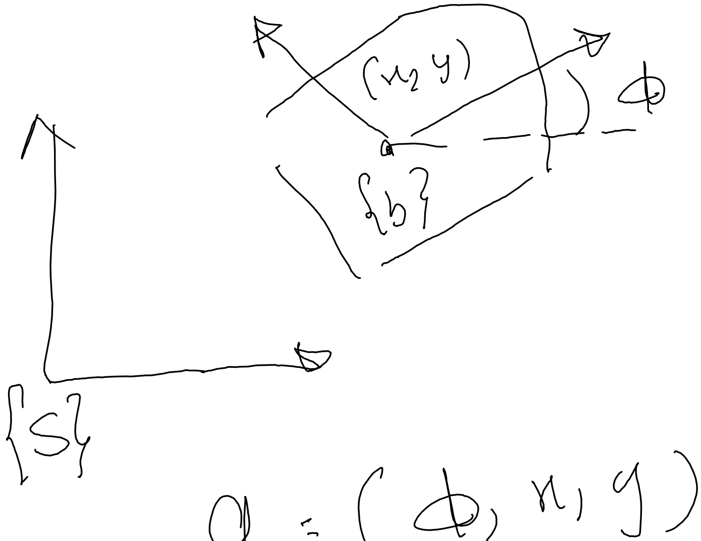
linear velocity @wheel in {b}

$$H(\phi) = \begin{bmatrix} h_1(\phi) \\ \vdots \\ h_m(\phi)\end{bmatrix} \in \mathbb{R}^{m\times 3} \Rightarrow u = H(\phi)\mathcal{V}_b$$

chassis twist $\mathcal{V}_b$

$$H(\phi) \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & \sin\phi \\ 0 & -\sin\phi & \cos\phi \end{bmatrix}$$

Configuration of chasis of wheeled Robot



$$q = (\phi, x, y)$$

$$u = H(\phi)\dot{q}$$

↓ wheel speed

$$\dot{q}(t) = \underbrace{\dot{q}_d(t)}_{\text{Fd}} + \underbrace{k_d\left(q_d(t) - q(t)\right) + k_i\int\left(q_d(t) - q(t)\right)dt}_{\text{feedback PI}}$$

$$\Rightarrow u = H(\phi)\mathcal{V} \Rightarrow -u_{i,max} \le u_i = h_i(\phi)\mathcal{V}_b \le u_{i,max}$$

Friction Compensation

Let's break this down into three clear sections: **Torque vs. Admittance**, **Friction Compensation (The "Static/Dynamic" part)**, and **Making it Transparent**.

1. The Core Difference: Torque Control vs. Admittance Control

Think of these as two different "personalities" you can give the robot.

Torque Control (Force Control)

- **Input:** You command a specific Torque (τ_{cmd}).
- **Behavior:** The robot acts like a **muscle**. It pushes with a specific force, regardless of whether it is moving or blocked by a wall.
- **Hardware Requirement:** This requires the motor to be "transparent" (back-drivable) naturally, OR you need excellent friction compensation so the motor doesn't "fight" the movement.
- **Equation:** $\tau_{motor} = \tau_{desired}$

Admittance Control (Position/Velocity Control based on Force)

- **Input:** You measure the **Interaction Force** (F_{ext}) from the user using a sensor.
- **Behavior:** The robot simulates a virtual object (like a mass-spring-damper). It calculates "If I were a 1kg mass and you pushed me with 2N, how fast would I move?" Then, it commands the inner velocity loop to go that fast.
- **Hardware Requirement:** It works well even on **non-back-drivable** motors (high gear ratios) because the inner velocity loop fights the friction for you.
- **Equation:** $M\ddot{x} + B\dot{x} + Kx = F_{ext}$
    - Output $\rightarrow$ Reference Velocity for the motor.

2. Friction Compensation: How to make it "Transparent"

You asked specifically about friction compensation, sweeping velocities, and "Dither." Let's disentangle the two methods you described.

To make a non-back-drivable motor feel "transparent" (like it has zero friction), we must mathematically model the friction and add extra torque to cancel it out.

A. The Friction Model (Static + Dynamic)

You correctly described the identification process. Here is the standard way to find the friction parameters:

1. **The Experiment (Sweeping):**
    - You command the robot to move at various constant velocities (e.g., 0.1 rad/s, 0.5 rad/s, 1.0 rad/s).
    - **Frequency:** You mentioned "0.2 Hz". This is likely the frequency of the **sine wave velocity command** used to generate these motions during the test. It is *slow* so that we can ignore acceleration (inertia) and measure pure friction.
    - **Measure:** You measure the Torque required to maintain those speeds.
1. **The Plot (The Stribeck Curve):**
    - You plot **Velocity (X-axis)** vs. **Torque (Y-axis)**.
    - **Coulomb Friction (F_c):** This is the "offset." Even at 0.001 rad/s, you need a chunk of torque just to move.
    - **Viscous Friction (F_v):** This is the "slope." As you go faster, the oil/grease resists more. $F_v \times Velocity$.
1. **The Compensation:**
    - Once you know F_c and F_v, you add them to your controller:
    - $\tau_{motor} = \tau_{user\_command} + [F_c \cdot \text{sign}(\dot{q}) + F_v \cdot \dot{q}]$
    - Now, when the user pushes, the motor uses its own power to fight the grease and gears, so the user feels nothing.

B. What is "Dither"? (The High Frequency Signal)

You were confused about the "High Frequency at 0." This is a specific trick called **Dither** used to kill Static Friction (Stiction).

- **The Problem:** Static friction is nasty. When velocity is exactly 0, the gears "lock" or stick. It takes a big spike of force to break them loose, creating a "jerk" feeling.
- **The Solution (Dither):** You inject a high-frequency vibration signal (noise) into the torque command.
    - **Frequency:** When we say "high frequency," we mean a frequency **faster than the robot can physically move**. For a typical robot arm, this might be **50 Hz to 200 Hz**. The heavy robot arm acts as a low-pass filter—it doesn't actually shake visibly.
    - **Amplitude:** The amplitude is usually set just below the static friction limit (e.g., if Stiction is 7Nm, Dither might be ±6Nm).
- **The Intuition:** Imagine a heavy box on a carpet. It's hard to start pushing. But if you vibrate the carpet rapidly, the box never truly "settles" into the static hole. It is always "dancing" slightly, keeping it in the dynamic friction regime, making it slide easily immediately when pushed.

3. Admittance Control & Transparency (Your Last Question)

You asked: *"If we implement admittance control... and human applies force... is stiffness (K) low for transparency?"*

Yes, exactly.

In Admittance Control, you define the "Virtual Environment":

$$M_{virtual} \cdot \ddot{x} + B_{virtual} \cdot \dot{x} + K_{virtual} \cdot x = F_{measured}$$

To make the robot **Transparent (Zero Force Mode)**, you tune the parameters like this:

1. **Stiffness ($K \approx 0$):**
    - If K is high, the robot acts like a stiff spring and wants to return to center.
    - If K is 0, the robot has no "home" position. It goes wherever you push it.
1. **Damping (B is low):**
    - If B is high, the robot feels like it is moving through honey.
    - If B is low, the robot feels like it is moving through air.
1. **Mass (M is low):**
    - If M is low, the robot accelerates quickly with a light touch.

Why Admittance is great for Non-Back-Drivable Motors:

Even if your motor has huge friction (gears), the **Force Sensor** detects your finger pushing. The Admittance Controller calculates: *"Hey, the user pushed! According to my equation (low Mass/Damping), I should be moving at 2 m/s!"*

The controller then commands the **Velocity Loop** to go 2 m/s. The high-gain velocity loop pumps huge current into the motor to overcome the real gear friction and hit 2 m/s.

Result: You feel like you are pushing a feather, even though the motor is fighting massive gear friction in the background.

Summary Checklist for your Answer

If they ask you "How to make a non-backdrivable motor transparent?":

1. **Torque Control Approach:** "We must use **Feedforward Friction Compensation**. We identify the Coulomb and Viscous friction by sweeping velocities (at ~0.2Hz sine waves) and fitting a curve. We can also use a **Dither signal** (high frequency ~100Hz vibration) to break static friction at zero velocity."
2. **Admittance Control Approach:** "We use a force sensor to measure user intent. We set the virtual stiffness (K) to zero and virtual damping (B) to a low value. The inner velocity loop handles the actual physical friction, making the system feel massless to the user."

2. Gravity Compensation: Is it included?

Answer: NO. Gravity is completely separate from friction.

- **Friction** depends on **Velocity** ($\dot{q}$). (If you stop moving, friction disappears—except for stiction).
- **Gravity** depends on **Position** (q). (If you stop moving, gravity is *strongest* because you still have to hold the arm up).

How do we identify Gravity Compensation?

Since gravity depends on position, not speed, we identify it using a **Static Experiment** (opposite of the friction sweep).

1. **The Experiment:**
    - Move the robot to a position (e.g., arm straight out horizontally). **Stop completely.**
    - Record the torque required to hold it there.
    - Move to a new angle (e.g., 45 degrees). Stop. Record Torque.
    - Repeat for many angles.
1. **The Math ($G(q)$):**
    - You fit a curve based on the physics: $\tau_g = m \cdot g \cdot r \cdot \sin(\theta)$
    - Where m is mass, r is the center of mass distance.
1. **The Result:**
    - You get a function $\hat{G}(q)$.
    - Now, your controller constantly calculates: *"The arm is at 90 degrees, gravity is pulling down with 10Nm. I will command +10Nm up."*
    - **Result:** The arm feels "weightless." If you let go, it floats in space rather than falling.

3. The "Perfect" Transparent Controller Equation

If Amazon asks you to write the control law for a **fully transparent robot** (Impedance/Admittance), here is the full equation that combines everything you just asked about.

$$\tau_{motor} = \tau_{interaction} + \overbrace{G(q)} + \overbrace{F_c\, sgn(v) + F_v \dot{q}} + \tau_{dither}$$

1. **Gravity ($\hat{G}(q)$):** Cancels the weight of the arm. (Depends on Position).
2. **Friction ($\hat{F}_c, \hat{F}_v$):** Cancels the gear drag. (Depends on Velocity).
3. **Dither:** Cancels the "stickiness" at zero.
4. **Interaction:** This is what makes it move based on the user's touch.

Interview Pro-Tip:

When identifying these, **do Gravity FIRST.**

Why? Because gravity is huge. If you try to measure friction while gravity is pulling down on the arm, your friction data will be noisy and wrong.

1. **Step 1:** Identify Gravity (Static test). Subtract it.
2. **Step 2:** Identify Friction (Velocity sweep), using the gravity-compensated torque.

1. Friction & Stiction (Low-Level Identification)

A. Stiction (Static Friction)

- **Definition:** The force required to "break" the static hold when velocity is exactly 0. It is the "stickiness" that causes a jerk at the start of motion.
- **How to Identify:**
    - **Signal:** Apply a very slow ramp command (Quasi-static) or very low-frequency sine wave (≈ 0.01 Hz).
    - **Measure: Motor Torque** (via Current $\times K_t$). *Do not use an external force sensor; you want to measure the internal gear friction.*
    - **Result:** The torque value just before the motor actually moves is your Stiction Torque.
- **Dither Signal (The Stiction Killer):**
    - **What is it?** A high-frequency vibration added to the torque command to keep the gears "dancing" so they never settle into the static hole.
    - **Frequency:** Higher than the system bandwidth (typically **50 Hz – 200 Hz**).
    - **Amplitude:** Just below the static friction limit (e.g., if Stiction is 5Nm, Dither is $\pm 4Nm$).

B. Dynamic Friction (Coulomb + Viscous)

- **Definition:** The resistance while moving.
    - **Coulomb:** Constant rubbing (Slide friction).
    - **Viscous:** Fluid drag from oil/grease (Increases with speed).
- **The Stribeck Curve:** A plot of **Velocity (X-axis)** vs. **Friction Torque (Y-axis)**.
- **How to Identify:**
    - **Signal:** Command **Constant Velocities** (Step inputs). E.g., Move at 0.1 rad/s, wait to settle, measure torque. Then 0.5 rad/s, etc.
    - **Why not Chirp?** A Chirp (Frequency Sweep) is for *Inertia/Bandwidth*. For friction, we want constant speed to eliminate acceleration ($J \cdot \alpha = 0$).
- **Compensation Equation:**

$$\tau_{comp} = F_c \cdot \text{sign}(\dot{q}) + F_v \cdot \dot{q}$$

Gravity Compensation

- **When to use:** Whenever a robot link moves **vertically** (against gravity).
- **When to ignore:** If the robot moves purely on a horizontal plane (e.g., a planar arm on a table), gravity is perpendicular to motion and doesn't create torque on the joint (though it creates normal force/friction).
- **For your Exoskeleton:**
    - **Swing Phase:** Yes, gravity pulls the leg down.
    - **Stance Phase:** The structure bears the weight, but gravity comp is still needed if you want the joints to feel "loose."

Admittance vs. Torque Control (Transparency)

Why Admittance is better for Non-Backdrivable Gears:

- **Torque Control + Friction Comp:** Relies on a mathematical model. If the model is 95% accurate, the user still feels the 5% error (stickiness). It is "Open Loop" regarding friction.
- **Admittance Control:** Uses a **High-Gain Velocity Loop**.
    - The Admittance law converts User Force $\rightarrow$ Desired Velocity.
    - The inner PID loop screams: *"I must reach this velocity!"* and automatically dumps as much current as needed to break friction.
    - It **hides** the friction inside the PID loop, making it robust to modeling errors.

Bandwidth & Bode Plots

A. How to Plot (The Math)

- **Input:** Sine wave of amplitude A (at frequency f).
- **Output:** Sine wave of amplitude B.
- **Magnitude (dB):** $20 \cdot \log_{10}(B / A)$.
- **Phase (Degrees):** Measure time delay Δt between peaks. Phase $= -(\Delta t \times f \times 360)$.

B. Definitions

- **Bandwidth:** The frequency where Magnitude drops to **-3 dB** ($0.707 \times \text{Input}$). This is the "Speed Limit" of the system.
- **Natural Frequency:** The frequency where the Magnitude **Peaks** (Resonance).

C. Which Loop?

- **Current Loop:** Lock the motor. (Frequency: > 500 Hz).
- **Velocity Loop:** Motor is free to spin. (Frequency: > 5-10 Hz).
- **Your Paper:** Characterized **Human + Robot** velocity bandwidth. This proved the *mechanical* system is fast enough (5 Hz) to keep up with human balance recovery.

D. Why Lock the Motor for Current Bandwidth?

- To measure pure electrical dynamics (L/R). If the motor spins, **Back-EMF** acts as a disturbance voltage, corrupting the measurement of how fast the amplifier can drive current.

Effective Inertia (J_{eff})

A. Measurement (Frequency Domain - The Best Way)

- Perform a high-frequency chirp (torque command).
- Look at the Bode Plot (Velocity/Torque).
- At high freq (>10Hz), the slope is -20dB/dec.
- **Formula:** $J = 1 / (\text{Magnitude} \cdot \omega)$
- *Note: Magnitude is absolute value, not dB. ω is rad/s.*

B. Measurement (Time Domain - The "Interview" Way)

- Apply a Step Torque (τ). Measure Acceleration (α).
- **Formula:** $J = \tau / \alpha$.
- **Why ignore Friction (B)?**
    - Equation: $\tau = J\alpha + B\omega$.
    - At the very instant you start ($t=0^+$), Velocity (ω) is **zero**.
    - Therefore, $B\omega = 0$. All torque goes into accelerating the Inertia.

C. Compensation:

- Calculate $\tau_{ff} = J_{eff} \cdot \ddot{q}_{desired}$.
- Add this to your command to help the user accelerate the heavy motor rotor.

Motor Constants

- **Torque Constant (K_t):**
    - Converts **Current (Amps) $\rightarrow$ Torque (Nm)**.
    - Used in the controller to translate "Desired Torque" into "Commanded Current."
- **Back-EMF Constant (K_e):**
    - Converts **Speed (rad/s) $\rightarrow$ Voltage (Volts)**.
    - "Generator Effect": The faster you spin, the more voltage the motor pushes back at you.
    - Used for **Feedforward Voltage Compensation** (to cancel the push-back).

① High-level → admittance

② → $\dot{q}_{cmd} = \frac{1}{M(s)+B}$

③ $\tau_{feedback} = k_p(\underbrace{error}_{\dot{q}_{desired} - \dot{q}_{measured}}) + k_i \int (error)$

④ $\tau_{FF} = G(q) + F_{friction} + J_{eff}\,\ddot{q}$

$\tau_{total} = \tau_{fd} + \tau_{FF}$

⑤ $I_{des} = \frac{\tau_{total}}{k_t}$

⑥ $V_{motor} = PID(\underbrace{I_{err}}_{I_{des} - I_{measured}}) + \underbrace{k_e \cdot \omega}_{\text{Back-EMF Compensator}}$